

# MediFlow — Hospital Management System

## Complete Technical Documentation

### 1. Project Overview

#### Purpose

MediFlow is a comprehensive **Hospital Management System (HMS)** frontend application designed to digitize and streamline hospital operations. It provides role-based dashboards and management tools for administrators, doctors, and receptionists.

#### Problem It Solves

- **Manual processes:** Replaces paper-based patient records, appointment scheduling, and queue management
- **Coordination gaps:** Provides real-time visibility into doctor schedules, patient queues, and appointment statuses
- **Data accessibility:** Centralizes patient medical records, vitals, medications, and treatment notes
- **Operational efficiency:** Automates appointment booking, patient check-in, and status tracking

#### Main Features

| Feature                  | Description                                                      |
|--------------------------|------------------------------------------------------------------|
| Role-Based Dashboards    | Separate dashboards for Admin, Doctor, and Receptionist          |
| Patient Management       | Full CRUD for patient records, medical info, vitals, medications |
| Appointment Scheduling   | Book, edit, cancel, and track appointments by type               |
| Queue Management         | Real-time patient queue with check-in and status advancement     |
| Doctor Scheduling        | Create and manage doctor availability and time slots             |
| Staff Management         | Admin can create doctor and receptionist accounts                |
| Pending Account Approval | Admin approves or rejects new staff registrations                |
| Activity Reports         | Admin-level statistics and reporting                             |
| Password Recovery        | Forgot/reset password flow                                       |
| JWT Authentication       | Secure access and refresh token mechanism                        |

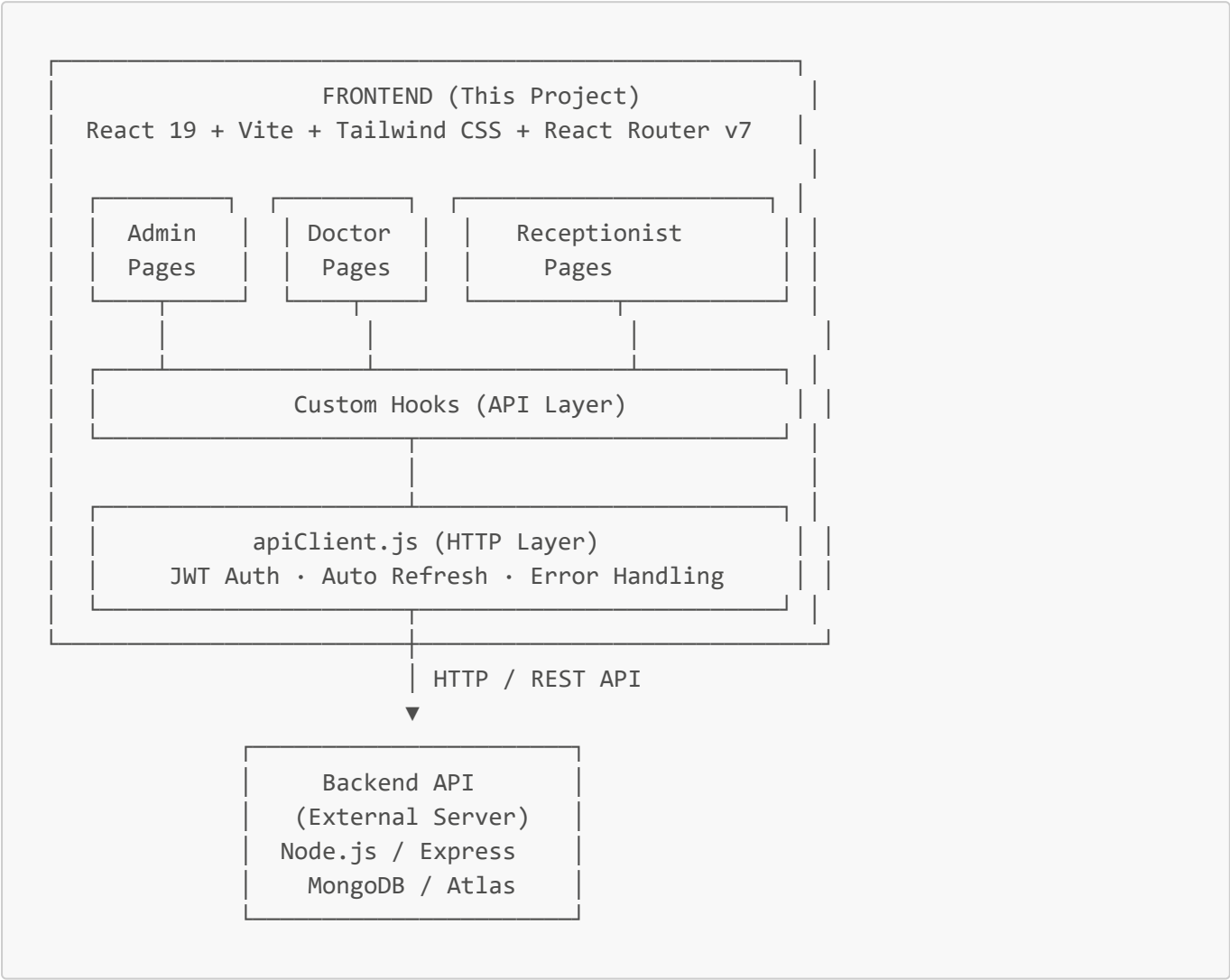
#### Target Users

- **Hospital Administrators** — System configuration, staff management, reports
- **Doctors** — View appointments, manage patient records, update medical information

- **Receptionists** — Register patients, book appointments, manage queue, handle billing

## 2. System Architecture

### High-Level Architecture



### Design Patterns

| Pattern                  | Usage                                                             |
|--------------------------|-------------------------------------------------------------------|
| Custom Hooks Pattern     | Business logic and API calls abstracted into reusable hooks       |
| Container/Presentational | Pages handle logic, components handle UI rendering                |
| Protected Route Pattern  | Role-based route guards via <code>ProtectedRoute</code> component |
| Configuration-Driven UI  | Stats cards and sidebar menus driven by config arrays             |
| API Client Abstraction   | Centralized HTTP client with JWT interceptor pattern              |
| Context API              | Auth context for global user/token state management               |

### Request Lifecycle

1. **User Action** → Component calls a custom hook function
2. **Hook** → Calls `apiRequest()` from `apiClient.js`
3. **apiClient** → Attaches JWT access token to `Authorization` header
4. **HTTP Request** → Sent to backend API
5. **401 Response** → `apiClient` attempts token refresh using refresh token
6. **Retry** → Original request is retried with new access token
7. **Refresh Failure** → User is logged out and redirected to `/login`
8. **Success** → Response data is returned to hook → state updated → UI re-renders

### 3. Technologies Used

| Technology             | Purpose                 | Why                                                      |
|------------------------|-------------------------|----------------------------------------------------------|
| React 19               | UI library              | Component-based architecture, hooks, concurrent features |
| Vite                   | Build tool & dev server | Fast HMR, optimized builds, ES module support            |
| Tailwind CSS 4         | Utility-first CSS       | Rapid UI development, consistent design system           |
| React Router v7        | Client-side routing     | SPA navigation, nested routes, route guards              |
| React Icons (Lucide)   | Icon library            | Consistent icon set with tree-shaking                    |
| Recharts               | Data visualization      | Lightweight React charting for dashboards                |
| Context API            | State management        | Built-in React state for auth context                    |
| JWT (Access + Refresh) | Authentication          | Stateless auth with automatic token refresh              |
| REST API               | Backend communication   | Standard HTTP methods, JSON payloads                     |

### 4. Folder Structure

```
final-project/
├── index.html           # Vite entry HTML
├── package.json         # Dependencies and scripts
├── vite.config.js       # Vite configuration
├── eslint.config.js     # ESLint configuration
├── TODO.md              # Development task tracker
├── data.text            # Notes / reference data
├──
├── public/
│   ├── favicon.svg      # Application favicon
│   ├── icons.svg        # SVG icon sprite
│   └── imgs/            # Static images
```

```

└─ src/
  └─ main.jsx          # React DOM entry point
  └─ App.jsx           # Root component with routing
  └─ index.css         # Global styles (Tailwind + custom)

  └─ assets/           # Static assets (images, etc.)

  └─ components/       # Reusable UI components
    └─ DashboardLayout.jsx # Shell layout (sidebar + header + outlet)
    └─ Sidebar.jsx      # Navigation sidebar (role-based menus)
    └─ Header.jsx       # Top header bar with user info
    └─ ProtectedRoute.jsx # Auth/role route guard
    └─ StateDoctor.jsx  # Statistics card widget
    └─ PatientManagement.jsx # Patient table component
    └─ AppointmentManagement.jsx # Appointment table component
    └─ CalendarComponent.jsx # Calendar view component
    └─ PatientInformation.jsx # Patient details wrapper
    └─ PatientInfo.jsx  # Patient personal info card
    └─ MedicalInfo.jsx  # Medical conditions display
    └─ EmergencyContact.jsx # Emergency contact card
    └─ PatientAppointment.jsx # Patient appointment list
    └─ Vitals.jsx       # Patient vitals display/edit
    └─ Medications.jsx  # Medications list with add/edit
    └─ PatientNotes.jsx # Doctor notes for patients
    └─ HealthReports.jsx # Health report display
    └─ DoctorSetting.jsx # Doctor settings/profile
    └─ Reception/
      └─ ReceptionWidgets.jsx # Status badges, queue badges, notifications

  └─ config/           # Configuration arrays
    └─ statsConfig.js   # Doctor dashboard stats card config
    └─ receptionStatsConfig.js # Reception dashboard stats config

  └─ context/          # React Context providers
    └─ AuthContext.jsx  # Authentication context (user, tokens)

  └─ data/             # Static/mock data

  └─ hooks/            # Custom React hooks (API layer)
    └─ useDoctorStats.js   # Doctor dashboard statistics
    └─ usePatientDetails.js # Single patient data fetcher
    └─ usePatientAppointments.js # Patient's appointment list
    └─ useAllAppointments.js # All appointments for doctor
    └─ useUpdateVitals.js  # Update patient vitals
    └─ useUpdateMedication.js # Update patient medications
    └─ useUpdatePatientNotes.js # Update patient notes
    └─ useReceptionistStats.js # Reception dashboard statistics
    └─ useReceptionistPatients.js # Patient list with
search/filter/pagination
    └─ useReceptionistAppointments.js # Today's appointments (reception)
    └─ useReceptionistAllAppointments.js # All appointments with CRUD
    └─ useReceptionistQueue.js # Queue management with check-in/status
    └─ useLiveQueue.js       # Live queue data for dashboard

```

```
└─ useAdminCreateStaff.js # Admin staff creation

└─ pages/                                # Route-level page components
  └─ Login.jsx                          # Login page
  └─ ForgotPassword.jsx                  # Password reset page
  └─ Doctor/
    └─ Doctor_Dashboard.jsx # Doctor main dashboard
    └─ Doctor_Appointments.jsx # Doctor appointments view
    └─ DoctorPatientDetails.jsx # Doctor patient detail view
  └─ Reception/
    └─ Reception_Dashboard.jsx # Reception main dashboard
    └─ Reception_Appointments.jsx # Appointment management
    └─ Reception_QueueManagement.jsx # Queue management
    └─ Reception_PatientManagement.jsx # Patient registration/list
    └─ Reception_PatientDetails.jsx # Patient detail view
    └─ Reception_BookingRequests.jsx # Booking requests
  └─ Admin/
    └─ Admin_Dashboard.jsx # Admin main dashboard
    └─ Admin_UserManagement.jsx # User management
    └─ Admin_DoctorManagement.jsx # Doctor management
    └─ Admin_PatientManagement.jsx # Patient management
    └─ Admin_DoctorsSchedule.jsx # Doctor schedule management
    └─ Admin_CreateStaff.jsx # Create staff accounts
    └─ Admin_PendingAccounts.jsx # Pending account approvals
    └─ Admin_ActivityReports.jsx # Activity reports

└─ styles/                              # Additional CSS styles
  └─ calendar.css                        # Calendar component styles

└─ utils/                               # Utility functions
  └─ apiClient.js                       # HTTP client with JWT interceptor
  └─ auth.js                           # Auth token helpers (get, set, clear)
  └─ refreshToken.js                   # Token refresh logic
```

## 5. Environment Variables

| Variable                   | Required | Purpose                    | Example                                                                       |
|----------------------------|----------|----------------------------|-------------------------------------------------------------------------------|
| VITE_API_BASE_URL          | Yes      | Backend API base URL       | <a href="http://localhost:5000/api">http://localhost:5000/api</a>             |
| VITE_DEFAULT_AVATAR_URL    | No       | Default avatar placeholder | <a href="https://example.com/avatar.png">https://example.com/avatar.png</a>   |
| VITE_DEFAULT_PATIENT_PHOTO | No       | Default patient photo      | <a href="https://example.com/patient.png">https://example.com/patient.png</a> |
| VITE_APP_NAME              | No       | Application display name   | MediFlow                                                                      |

| Variable             | Required | Purpose                 | Example                    |
|----------------------|----------|-------------------------|----------------------------|
| VITE_APP_VERSION     | No       | Application version     | 1.0.0                      |
| VITE_APP_DESCRIPTION | No       | Application description | Hospital Management System |

**Note:** Vite exposes only variables prefixed with `VITE_` to the client bundle.

## 6. Database Documentation

**This is a frontend-only project.** The database models are inferred from the API request/response structures observed in the frontend code. The actual backend uses MongoDB.

### Inferred Data Models

#### User

| Field                 | Type     | Description                                                                                 |
|-----------------------|----------|---------------------------------------------------------------------------------------------|
| <code>_id</code>      | ObjectId | Unique identifier                                                                           |
| <code>fullName</code> | String   | User's full name                                                                            |
| <code>email</code>    | String   | User email (unique)                                                                         |
| <code>password</code> | String   | Hashed password                                                                             |
| <code>gender</code>   | String   | <code>male</code>   <code>female</code>                                                     |
| <code>age</code>      | Number   | User's age                                                                                  |
| <code>phone</code>    | String   | Phone number                                                                                |
| <code>address</code>  | String   | Physical address                                                                            |
| <code>role</code>     | String   | <code>admin</code>   <code>doctor</code>   <code>receptionist</code>   <code>patient</code> |
| <code>isActive</code> | Boolean  | Account active status                                                                       |
| <code>avatar</code>   | String   | Profile image URL                                                                           |

#### Doctor (extends User)

| Field                                                | Type                  | Description             |
|------------------------------------------------------|-----------------------|-------------------------|
| <code>userId</code>                                  | ObjectId (ref → User) | Associated user account |
| <code>specialty</code> / <code>specialization</code> | String                | Medical specialty       |
| <code>schedule</code>                                | Array                 | Available time slots    |

## Patient (extends User)

| Field                          | Type                    | Description                                                                                                                                                                                   |
|--------------------------------|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>userId</code>            | ObjectId (ref → User)   | Associated user account                                                                                                                                                                       |
| <code>patientId</code>         | String                  | Auto-generated patient code                                                                                                                                                                   |
| <code>dateOfBirth</code>       | Date                    | Date of birth                                                                                                                                                                                 |
| <code>occupation</code>        | String                  | Patient's occupation                                                                                                                                                                          |
| <code>bloodType</code>         | String                  | Blood type ( <code>A+</code> , <code>O-</code> , etc.)                                                                                                                                        |
| <code>insuranceProvider</code> | String                  | Insurance company                                                                                                                                                                             |
| <code>insuranceClass</code>    | String                  | Insurance tier/class                                                                                                                                                                          |
| <code>patientType</code>       | String                  | <code>outpatient</code>   <code>inpatient</code>                                                                                                                                              |
| <code>currentStatus</code>     | String                  | <code>inTreatment</code>   <code>admitted</code>   <code>discharged</code>                                                                                                                    |
| <code>doctorId</code>          | ObjectId (ref → Doctor) | Assigned primary doctor                                                                                                                                                                       |
| <code>medicalInfo</code>       | Object                  | Medical conditions, allergies, etc.                                                                                                                                                           |
| <code>emergencyContact</code>  | Object                  | { <code>name</code> , <code>phone</code> , <code>relation</code> }                                                                                                                            |
| <code>vitals</code>            | Object                  | { <code>bp</code> , <code>temperature</code> , <code>heartRate</code> , <code>weight</code> , <code>height</code> , <code>spO2</code> , <code>respiratoryRate</code> , <code>glucose</code> } |
| <code>medications</code>       | Array                   | List of prescribed medications                                                                                                                                                                |
| <code>notes</code>             | Array                   | Doctor notes with timestamps                                                                                                                                                                  |

## Appointment

| Field                        | Type                     | Description                                                              |
|------------------------------|--------------------------|--------------------------------------------------------------------------|
| <code>_id</code>             | ObjectId                 | Unique identifier                                                        |
| <code>patientId</code>       | ObjectId (ref → Patient) | Patient reference                                                        |
| <code>doctorId</code>        | ObjectId (ref → Doctor)  | Doctor reference                                                         |
| <code>appointmentType</code> | String                   | <code>consultation</code>   <code>followUp</code>   <code>surgery</code> |
| <code>date</code>            | Date                     | Appointment date                                                         |
| <code>startTime</code>       | String                   | Start time (HH:MM)                                                       |
| <code>endTime</code>         | String                   | End time (HH:MM)                                                         |

| Field  | Type   | Description                                             |
|--------|--------|---------------------------------------------------------|
| status | String | scheduled   waiting   withDoctor   completed   canceled |
| notes  | String | Appointment notes                                       |

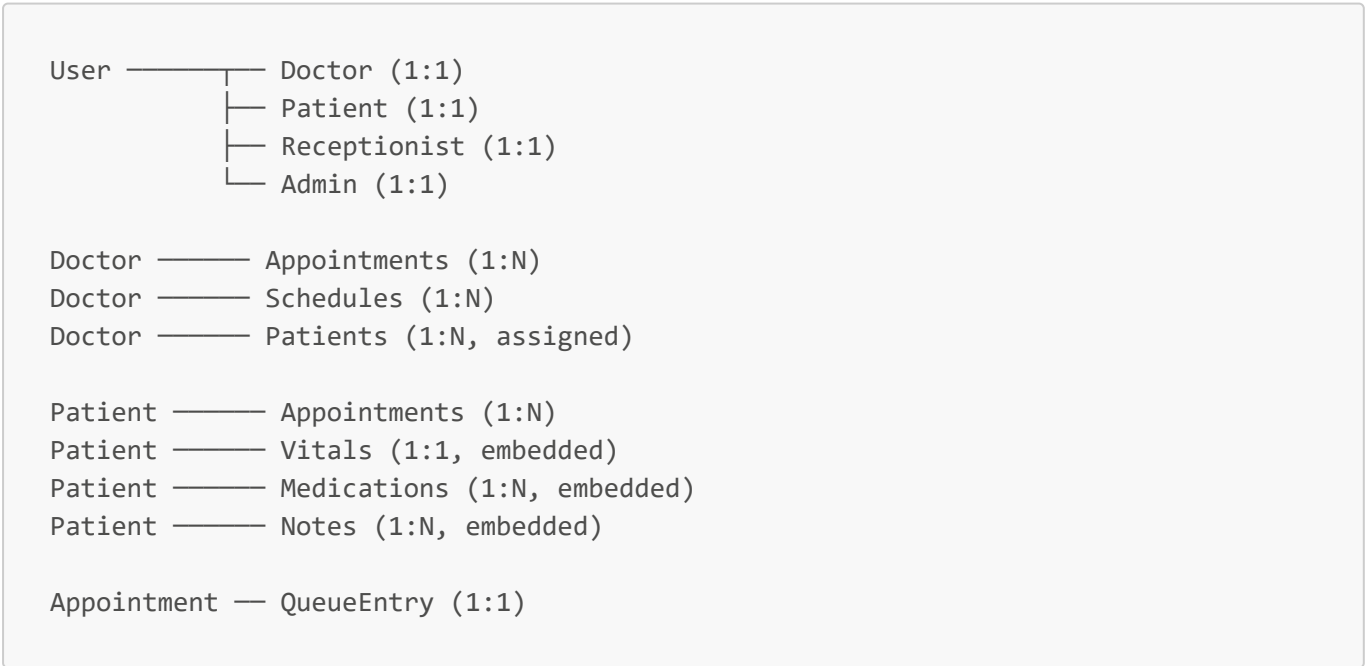
QueueEntry

| Field         | Type                         | Description                       |
|---------------|------------------------------|-----------------------------------|
| _id           | ObjectId                     | Unique identifier                 |
| appointmentId | ObjectId (ref → Appointment) | Associated appointment            |
| patientId     | ObjectId (ref → Patient)     | Patient reference                 |
| status        | String                       | Waiting   With Doctor   Completed |
| queueNumber   | Number                       | Position in queue                 |

DoctorSchedule

| Field       | Type                    | Description       |
|-------------|-------------------------|-------------------|
| doctorId    | ObjectId (ref → Doctor) | Doctor reference  |
| date        | Date                    | Schedule date     |
| startTime   | String                  | Slot start time   |
| endTime     | String                  | Slot end time     |
| isAvailable | Boolean                 | Slot availability |

Relationships



---

## 7. API Documentation

---

**Base URL:** Configured via `VITE_API_BASE_URL` (default: `http://localhost:5000/api`)

### Authentication Endpoints

#### POST `/auth/login`

| Property         | Value                                                                                                          |
|------------------|----------------------------------------------------------------------------------------------------------------|
| Description      | Authenticate user and receive tokens                                                                           |
| Auth Required    | No                                                                                                             |
| Request Body     | <code>{ "email": "string", "password": "string" }</code>                                                       |
| Success Response | <code>{ "success": true, "data": { "user": {...}, "accessToken": "string", "refreshToken": "string" } }</code> |
| Error Response   | <code>{ "success": false, "message": "Invalid credentials" }</code>                                            |

#### POST `/auth/refresh-token`

| Property         | Value                                                                 |
|------------------|-----------------------------------------------------------------------|
| Description      | Refresh expired access token                                          |
| Auth Required    | No (uses refresh token)                                               |
| Request Body     | <code>{ "refreshToken": "string" }</code>                             |
| Success Response | <code>{ "success": true, "data": { "accessToken": "string" } }</code> |

#### POST `/auth/forgot-password`

| Property      | Value                              |
|---------------|------------------------------------|
| Description   | Send password reset email          |
| Auth Required | No                                 |
| Request Body  | <code>{ "email": "string" }</code> |

#### POST `/auth/reset-password`

| Property      | Value                                                       |
|---------------|-------------------------------------------------------------|
| Description   | Reset password with token                                   |
| Auth Required | No                                                          |
| Request Body  | <code>{ "token": "string", "newPassword": "string" }</code> |

---

## Doctor Endpoints

### GET /doctors

| Property         | Value                                                     |
|------------------|-----------------------------------------------------------|
| Description      | List all doctors                                          |
| Auth Required    | Yes                                                       |
| Query Params     | <code>limit</code> (number), <code>search</code> (string) |
| Success Response | <code>{ "success": true, "data": [Doctor] }</code>        |

### GET /doctors/stats

| Property         | Value                                                                                         |
|------------------|-----------------------------------------------------------------------------------------------|
| Description      | Doctor dashboard statistics                                                                   |
| Auth Required    | Yes (Doctor role)                                                                             |
| Success Response | <code>{ "success": true, "data": { "total", "scheduled", "withDoctor", "completed" } }</code> |

### GET /doctors/appointments

| Property      | Value                     |
|---------------|---------------------------|
| Description   | Get doctor's appointments |
| Auth Required | Yes (Doctor role)         |

### GET /doctors/patients/:id

| Property      | Value                          |
|---------------|--------------------------------|
| Description   | Get patient details for doctor |
| Auth Required | Yes (Doctor role)              |
| Params        | <code>id</code> — Patient ID   |

### PATCH /doctors/patients/:id/vitals

| Property      | Value                                                                                |
|---------------|--------------------------------------------------------------------------------------|
| Description   | Update patient vitals                                                                |
| Auth Required | Yes (Doctor role)                                                                    |
| Request Body  | <code>{ "bp": "string", "temperature": "string", "heartRate": "string", ... }</code> |

## PATCH /doctors/patients/:id/medications

| Property      | Value                                                           |
|---------------|-----------------------------------------------------------------|
| Description   | Update patient medications                                      |
| Auth Required | Yes (Doctor role)                                               |
| Request Body  | <pre>{ "medications": [{ "name", "dose", "frequency" }] }</pre> |

## PATCH /doctors/patients/:id/notes

| Property      | Value                            |
|---------------|----------------------------------|
| Description   | Add/update patient notes         |
| Auth Required | Yes (Doctor role)                |
| Request Body  | <pre>{ "notes": "string" }</pre> |

## Receptionist Endpoints

### GET /receptionist/stats

| Property         | Value                                                                                                                                                                                                                                        |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Description      | Receptionist dashboard statistics                                                                                                                                                                                                            |
| Auth Required    | Yes (Receptionist role)                                                                                                                                                                                                                      |
| Success Response | <pre>{ "success": true, "data": { "todayAppointments", "completedToday", "inQueue", "totalPatients", "pendingBills", "todayRevenue", "withDoctor", "cancelledToday", "currentlyAdmitted", "appointments", "queue", "notifications" } }</pre> |

### GET /receptionist/today

| Property         | Value                                                                    |
|------------------|--------------------------------------------------------------------------|
| Description      | Today's appointments grouped by doctor                                   |
| Auth Required    | Yes (Receptionist role)                                                  |
| Success Response | <pre>{ "success": true, "data": { "Doctor Name": [Appointment] } }</pre> |

### GET /receptionist/patients

| Property      | Value                      |
|---------------|----------------------------|
| Description   | List patients with filters |
| Auth Required | Yes (Receptionist role)    |

| Property         | Value                                                                                  |
|------------------|----------------------------------------------------------------------------------------|
| Query Params     | <code>search, gender, type, status, page, limit</code>                                 |
| Success Response | <code>{ "success": true, "data": [Patient], "pagination": { "total": number } }</code> |

## POST `/receptionist/patients`

| Property      | Value                                                                                                                                                                                                                                      |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Description   | Register a new patient                                                                                                                                                                                                                     |
| Auth Required | Yes (Receptionist role)                                                                                                                                                                                                                    |
| Request Body  | <code>{ "fullName", "email", "password", "gender", "age", "dateOfBirth", "phone", "address", "occupation", "bloodType", "insuranceProvider", "insuranceClass", "patientType", "emergencyContact": { "name", "phone", "relation" } }</code> |

## GET `/receptionist/appointments`

| Property         | Value                                                                                            |
|------------------|--------------------------------------------------------------------------------------------------|
| Description      | List all appointments with filters                                                               |
| Auth Required    | Yes (Receptionist role)                                                                          |
| Query Params     | <code>status, date, page, limit</code>                                                           |
| Success Response | <code>{ "success": true, "data": [Appointment], "pagination": { "totalPages", "total" } }</code> |

## POST `/receptionist/appointments`

| Property      | Value                                                                                                |
|---------------|------------------------------------------------------------------------------------------------------|
| Description   | Book a new appointment                                                                               |
| Auth Required | Yes (Receptionist role)                                                                              |
| Request Body  | <code>{ "patientId", "doctorId", "appointmentType", "date", "startTime", "endTime", "notes" }</code> |

## PUT `/receptionist/appointments/:id`

| Property    | Value                          |
|-------------|--------------------------------|
| Description | Update an existing appointment |

| Property             | Value                                                                                                |
|----------------------|------------------------------------------------------------------------------------------------------|
| <b>Auth Required</b> | Yes (Receptionist role)                                                                              |
| <b>Params</b>        | <code>id</code> — Appointment ID                                                                     |
| <b>Request Body</b>  | <code>{ "patientId", "doctorId", "appointmentType", "date", "startTime", "endTime", "notes" }</code> |

PATCH `/receptionist/appointments/:id/cancel`

| Property             | Value                            |
|----------------------|----------------------------------|
| <b>Description</b>   | Cancel an appointment            |
| <b>Auth Required</b> | Yes (Receptionist role)          |
| <b>Params</b>        | <code>id</code> — Appointment ID |

GET `/receptionist/queue`

| Property                | Value                                                                                      |
|-------------------------|--------------------------------------------------------------------------------------------|
| <b>Description</b>      | Get all doctor queues                                                                      |
| <b>Auth Required</b>    | Yes (Receptionist role)                                                                    |
| <b>Success Response</b> | <code>{ "success": true, "data": [{ "doctorId": "...", "entries": [QueueEntry] }] }</code> |

GET `/receptionist/queue/:doctorId`

| Property                | Value                                                  |
|-------------------------|--------------------------------------------------------|
| <b>Description</b>      | Get queue for a specific doctor                        |
| <b>Auth Required</b>    | Yes (Receptionist role)                                |
| <b>Success Response</b> | <code>{ "success": true, "data": [QueueEntry] }</code> |

PATCH `/receptionist/queue/:appointmentId/checkin`

| Property             | Value                       |
|----------------------|-----------------------------|
| <b>Description</b>   | Check in a patient to queue |
| <b>Auth Required</b> | Yes (Receptionist role)     |

PATCH `/receptionist/queue/:appointmentId/status`

| Property           | Value                        |
|--------------------|------------------------------|
| <b>Description</b> | Advance patient queue status |

| Property      | Value                   |
|---------------|-------------------------|
| Auth Required | Yes (Receptionist role) |

## Admin Endpoints

POST `/admin/doctors`

| Property      | Value                                                              |
|---------------|--------------------------------------------------------------------|
| Description   | Create a new doctor account                                        |
| Auth Required | Yes (Admin role)                                                   |
| Request Body  | <code>{ "fullName", "email", "password", "specialty", ... }</code> |

POST `/admin/receptionists`

| Property      | Value                                                 |
|---------------|-------------------------------------------------------|
| Description   | Create a new receptionist account                     |
| Auth Required | Yes (Admin role)                                      |
| Request Body  | <code>{ "fullName", "email", "password", ... }</code> |

**Note:** Additional admin endpoints for user management, pending accounts, activity reports, doctor management, and schedule management are consumed by the admin pages but the exact endpoint signatures are determined by the backend API.

## 8. Authentication & Authorization

### Login Flow

1. User submits email + password on Login page
2. POST `/auth/login` → Backend validates credentials
3. Backend returns `{ user, accessToken, refreshToken }`
4. Tokens stored in `localStorage` via `auth.js` helpers
5. User object stored in `AuthContext` (React Context)
6. User redirected to role-specific dashboard:
  - admin → `/admin/dashboard`
  - doctor → `/doctor/dashboard`
  - receptionist → `/reception/dashboard`

### JWT Usage

| Token         | Storage      | Lifetime                   | Purpose                   |
|---------------|--------------|----------------------------|---------------------------|
| Access Token  | localStorage | Short-lived (e.g., 15 min) | Authenticate API requests |
| Refresh Token | localStorage | Long-lived (e.g., 7 days)  | Obtain new access tokens  |

## Token Attachment

Every API request through `apiClient.js`:

```
Authorization: Bearer <accessToken>
```

## Automatic Token Refresh

1. API request returns 401 Unauthorized
2. `apiClient` intercepts the error
3. Calls `POST /auth/refresh-token` with `refreshToken`
4. On success: updates stored `accessToken`, retries original request
5. On failure: clears all tokens, redirects to `/login`

## Password Hashing

- Handled on the **backend** (bcrypt)
- Frontend sends plaintext password over HTTPS
- No passwords stored in frontend

## Roles

| Role         | Access Level                                       |
|--------------|----------------------------------------------------|
| admin        | Full system access, staff management, reports      |
| doctor       | Patient records, appointments, medical data        |
| receptionist | Patient registration, appointments, queue, billing |

## Protected Routes

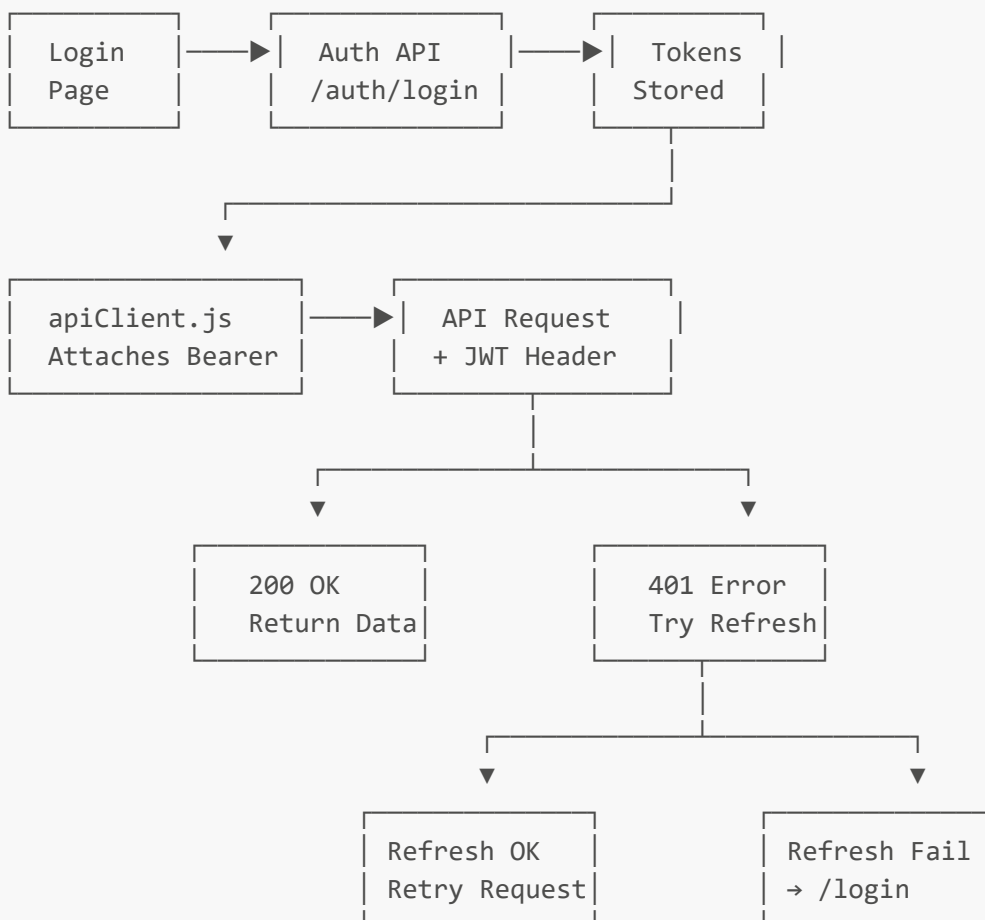
The `ProtectedRoute` component:

1. Checks if user is authenticated (token exists in context)
2. Verifies user role matches required role(s)
3. Redirects to `/login` if unauthorized
4. Redirects to appropriate dashboard if wrong role

## Middleware (Frontend Route Guards)

```
<Route element={<ProtectedRoute allowedRoles={["admin"]} />}>
  {/* Admin routes */}
</Route>
<Route element={<ProtectedRoute allowedRoles={["doctor"]} />}>
  {/* Doctor routes */}
</Route>
<Route element={<ProtectedRoute allowedRoles={["receptionist"]} />}>
  {/* Receptionist routes */}
</Route>
```

## Security Flow Diagram



## 9. Business Logic

### Module: Authentication ([Login.jsx](#), [ForgotPassword.jsx](#))

- Login validates email/password against backend
- On success, tokens are stored and user is context-persisted
- Forgot password sends email with reset link
- Auth context provides `user`, `isAuthenticated`, `login()`, `logout()` globally

## Module: Admin Management

- **User Management:** View, search, and manage all system users
- **Doctor Management:** View list of doctors, manage doctor profiles
- **Patient Management:** Admin-level patient record viewing
- **Create Staff:** Create new doctor or receptionist accounts via dedicated forms
- **Pending Accounts:** Review and approve/reject pending staff registrations
- **Doctor Schedules:** Create and manage doctor availability time slots
- **Activity Reports:** View system-wide statistics and activity data

## Module: Doctor Workflow

- **Dashboard:** Displays today's stats (appointments total, waiting, with doctor, completed)
- **Appointments:** Lists all assigned appointments with status tracking
- **Patient Details:** Full patient view with tabs for:
  - Personal info (name, age, gender, contact)
  - Medical info (conditions, allergies)
  - Emergency contacts
  - Vitals (BP, temperature, heart rate, weight, height, SpO2)
  - Medications (add/edit prescriptions)
  - Notes (clinical notes with timestamps)
  - Health reports
  - Appointment history

## Module: Receptionist Workflow

- **Dashboard:** Stats cards (today's appointments, total patients, in queue), quick actions, live queue, appointments table, notifications
- **Appointments:** Full CRUD — search, filter by status/date, paginate, book new, edit, cancel
- **Queue Management:** Real-time queue per doctor, check-in patients, advance status (Waiting → With Doctor → Completed)
- **Patient Management:** Register new patients with full demographic/insurance data, search, filter by gender/type/status
- **Booking Requests:** Manage incoming booking requests
- **Patient Details:** View comprehensive patient information

## Module: Queue Management

```
Patient arrives → Receptionist checks in → Status: "Waiting"
                                     ↓
Receptionist advances status → Status: "With Doctor"
                                     ↓
Doctor completes consultation → Status: "Completed"
```

---

## 10. Validation

---

## Frontend Validation

| Field      | Rules                                                      |
|------------|------------------------------------------------------------|
| Email      | Required, valid email format ( <code>type="email"</code> ) |
| Password   | Required, minimum 6 characters                             |
| Full Name  | Required                                                   |
| Phone      | Required, tel format                                       |
| Age        | Required, 0-150 range                                      |
| Date       | Required, valid date format                                |
| Time       | Required, valid time format (HH:MM)                        |
| Patient ID | Required for appointment booking                           |
| Doctor ID  | Required for appointment booking                           |

## Form-Level Validation

- All registration forms use HTML5 `required` attributes
- Custom validation in `handleSubmit` functions checks for required fields before API submission
- Error messages displayed inline below forms
- Submit buttons disabled during loading states

---

# 11. Middleware

---

## ProtectedRoute Component

**Execution:** Before any protected route renders

**Logic:**

1. Reads user from `AuthContext`
2. If no user/token → redirect to `/login`
3. If user role not in `allowedRoles` → redirect to appropriate dashboard
4. If authorized → render child routes via `<Outlet />`

## apiClient Interceptor

**Execution:** Before and after every API request

**Logic:**

1. **Pre-request:** Reads access token from `localStorage`, attaches to `Authorization` header
2. **Post-response:** If 401 error, attempts token refresh
3. **Refresh success:** Updates token, retries original request
4. **Refresh failure:** Clears auth data, redirects to `/login`

# DashboardLayout

**Execution:** Wraps all dashboard routes

**Logic:**

- 1. Renders **Sidebar** with role-based navigation
- 2. Renders **Header** with user info and logout
- 3. Renders child routes via `<Outlet />`

## 12. Error Handling

### Global Strategy

| Level          | Method                                                       |
|----------------|--------------------------------------------------------------|
| API Client     | Catches HTTP errors, handles 401 with token refresh          |
| Custom Hooks   | Try/catch blocks, store error in state, return to components |
| Components     | Display error messages in UI (red text, error banners)       |
| Forms          | Display validation and submission errors inline              |
| Loading States | Show loading indicators during async operations              |

### Error Response Format

```
{
  "success": false,
  "message": "Error description here"
}
```

### Error Handling Pattern (Hooks)

```
const [error, setError] = useState("");

try {
  const res = await apiRequest("/endpoint");
  const result = await res.json();
  if (!result.success) {
    setError(result.message);
    return;
  }
  // Handle success
} catch (err) {
  setError(`Something went wrong: ${err.message}`);
} finally {
```

```
setLoading(false);
}
```

## 13. File Uploads

**Current Status:** The frontend includes placeholder UI for health reports and profile photos, but no file upload implementation was found in the codebase. File upload functionality appears to be planned but not yet implemented.

### Observed UI Elements

- Patient photo placeholder with default avatar
- Health reports section in patient details (display only)

### Recommendations

- Implement file upload via `FormData` and `multipart/form-data`
- Support image uploads for patient/doctor profiles
- Support document uploads for health reports
- Add file type validation (jpg, png, pdf)
- Add file size limits (e.g., 5MB)

## 14. Security

### Implemented Security Measures

| Measure               | Implementation                                               |
|-----------------------|--------------------------------------------------------------|
| JWT Authentication    | Access + refresh token pair stored in localStorage           |
| Token Refresh         | Automatic refresh on 401 responses                           |
| Role-Based Access     | ProtectedRoute enforces role restrictions                    |
| Password Hashing      | Handled server-side (bcrypt)                                 |
| CORS                  | Configured on backend                                        |
| Input Validation      | HTML5 required fields + custom validation                    |
| HTTPS                 | Expected in production deployment                            |
| Environment Variables | Sensitive config via <code>.env</code> files (not committed) |

### Security Concerns & Recommendations

| Concern                      | Recommendation                                                |
|------------------------------|---------------------------------------------------------------|
| Tokens in localStorage       | Consider httpOnly cookies for XSS protection                  |
| No CSRF protection           | Implement CSRF tokens for state-changing requests             |
| No rate limiting on frontend | Backend should implement rate limiting                        |
| Console.log statements       | Remove debug <code>console.log</code> calls before production |
| No content security policy   | Add CSP headers                                               |
| No input sanitization        | Add DOMPurify or similar for user-generated content           |

## 15. Application Flow

### Startup Sequence

1. index.html loads
2. main.jsx renders `<App />` into DOM
3. App.jsx:
  - a. AuthContext wraps the entire app
  - b. React Router initializes routes
  - c. AuthContext checks for existing tokens in localStorage
  - d. If tokens exist → validate and restore user session
  - e. If no tokens → user is unauthenticated
4. Route matching:
  - "/" → redirect to /login
  - "/login" → Login page
  - "/forgot-password" → ForgotPassword page
  - "/admin/\*" → ProtectedRoute(admin) → DashboardLayout → Admin pages
  - "/doctor/\*" → ProtectedRoute(doctor) → DashboardLayout → Doctor pages
  - "/reception/\*" → ProtectedRoute(receptionist) → DashboardLayout → Reception pages

### User Session Flow

```
Login → Store tokens → Set user in context → Redirect to dashboard
├── Navigate app (protected routes check role)
├── API calls (apiClient attaches JWT)
│   ├── 200 → process data
│   └── 401 → refresh token → retry or logout
└── Logout → Clear tokens → Clear context → Redirect to /login
```

# 16. Deployment

---

## Local Setup

1. Clone the repository
2. Install dependencies: `npm install`
3. Create `.env` file with required variables
4. Start dev server: `npm run dev`

## Production Build

```
npm run build
```

Output is generated in `/dist` folder — a static SPA that can be served by any web server.

## Deployment Options

### Vercel / Netlify

1. Connect Git repository
2. Set build command: `npm run build`
3. Set output directory: `dist`
4. Add environment variables in dashboard
5. Deploy

### Railway

1. Connect Git repository
2. Railway auto-detects Vite project
3. Add environment variables
4. Deploy automatically on push

### Manual Server (Nginx)

```
server {  
    listen 80;  
    server_name your-domain.com;  
    root /path/to/dist;  
    index index.html;  
  
    location / {  
        try_files $uri $uri/ /index.html;  
    }  
}
```

## MongoDB Atlas Configuration

- Create cluster on MongoDB Atlas
- Whitelist server IP addresses
- Create database user with appropriate permissions
- Use connection string in backend `MONGODB_URI` environment variable

---

## 17. Installation Guide

---

### Prerequisites

- **Node.js** v18 or higher
- **npm** v9 or higher
- **Backend API** server running (separate repository)

### Steps

```
# 1. Clone the repository
git clone https://github.com/Nader-Gamal3011/Nader_Portfolio.git

# 2. Navigate to project directory
cd final-project

# 3. Install dependencies
npm install

# 4. Create environment file
copy .env.example .env      # Windows
cp .env.example .env        # macOS/Linux

# 5. Edit .env with your configuration
# VITE_API_BASE_URL=http://localhost:5000/api

# 6. Start development server
npm run dev
```

---

## 18. Running the Project

---

| Command                      | Description                                   |
|------------------------------|-----------------------------------------------|
| <code>npm install</code>     | Install all dependencies                      |
| <code>npm run dev</code>     | Start Vite development server (hot reload)    |
| <code>npm run build</code>   | Create production build in <code>/dist</code> |
| <code>npm run preview</code> | Preview production build locally              |

| Command                   | Description              |
|---------------------------|--------------------------|
| <code>npm run lint</code> | Run ESLint code analysis |

## 19. Project Features

### Feature Checklist

#### Authentication

- ☒ User login with email/password
- ☒ JWT access + refresh token system
- ☒ Automatic token refresh on 401
- ☒ Forgot password flow
- ☒ Role-based route protection
- ☒ Logout with session cleanup

#### Admin Module

- ☒ Admin dashboard with statistics
- ☒ User management
- ☒ Doctor management
- ☒ Patient management (view)
- ☒ Create staff accounts (doctors, receptionists)
- ☒ Pending account approval
- ☒ Doctor schedule management
- ☒ Activity reports

#### Doctor Module

- ☒ Doctor dashboard with statistics
- ☒ Appointment list and management
- ☒ Patient details view
- ☒ Update patient vitals
- ☒ Update patient medications
- ☒ Add patient notes
- ☒ Health reports view
- ☒ Calendar view
- ☒ Doctor settings/profile

#### Receptionist Module

- ☒ Receptionist dashboard with statistics
- ☒ Patient registration (full demographic data)
- ☒ Patient list with search, filter, pagination
- ☒ Appointment booking
- ☒ Appointment editing

- ☒ Appointment cancellation
- ☒ Appointment search and filters
- ☒ Real-time queue management
- ☒ Patient check-in
- ☒ Queue status advancement
- ☒ Quick action buttons
- ☒ Notification display

UI/UX

- ☒ Responsive sidebar navigation
- ☒ Role-based menu items
- ☒ Statistics card widgets
- ☒ Status badges with color coding
- ☒ Avatar with initials and color hashing
- ☒ Loading states and skeletons
- ☒ Error state displays
- ☒ Modal dialogs for forms
- ☒ Confirmation dialogs for destructive actions
- ☒ Pagination controls
- ☒ Search and filter toolbar

---

## 20. Future Improvements

---

High Priority

| Improvement         | Description                                                  |
|---------------------|--------------------------------------------------------------|
| Real-time Updates   | Implement WebSocket for live queue updates and notifications |
| File Uploads        | Support profile images, health report documents, lab results |
| Print Support       | Printable prescriptions, patient summaries, reports          |
| i18n / Localization | Multi-language support (Arabic/English)                      |
| Dark Mode           | Theme toggle for better accessibility                        |

Medium Priority

| Improvement          | Description                                             |
|----------------------|---------------------------------------------------------|
| Search Enhancement   | Global search across patients, doctors, appointments    |
| Notifications System | Real-time push notifications for appointments and queue |
| Billing Module       | Invoice generation, payment tracking, insurance claims  |
| Lab Results          | Integration with laboratory information system          |

| Improvement                  | Description                                   |
|------------------------------|-----------------------------------------------|
| <b>Prescription Print</b>    | Generate printable prescription documents     |
| <b>Appointment Reminders</b> | SMS/Email reminders for upcoming appointments |

## Low Priority

| Improvement                    | Description                                     |
|--------------------------------|-------------------------------------------------|
| <b>PWA Support</b>             | Progressive Web App for offline access          |
| <b>Data Export</b>             | Export reports as PDF/Excel                     |
| <b>Audit Logging</b>           | Track all data changes with user attribution    |
| <b>Dashboard Customization</b> | Configurable dashboard widgets per role         |
| <b>Advanced Analytics</b>      | Charts for patient trends, appointment patterns |

## 21. Code Quality Review

### Strengths

| Area                         | Assessment                                                                                                 |
|------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>Folder Organization</b>   | Well-structured with clear separation (pages, components, hooks, config, utils)                            |
| <b>Custom Hooks</b>          | Excellent abstraction of API logic into reusable hooks                                                     |
| <b>Component Reusability</b> | <code>StateDoctor</code> , <code>DashboardLayout</code> , <code>StatusBadge</code> are properly abstracted |
| <b>Configuration-Driven</b>  | Stats cards and menus use config arrays, reducing code duplication                                         |
| <b>Naming Conventions</b>    | Consistent PascalCase for components, camelCase for hooks/utils                                            |
| <b>API Abstraction</b>       | Clean <code>apiClient</code> with centralized auth handling                                                |
| <b>Role-Based Access</b>     | Clear role separation in routing and navigation                                                            |

### Areas for Improvement

| Area                  | Issue                                                  | Recommendation                                   |
|-----------------------|--------------------------------------------------------|--------------------------------------------------|
| <b>Debug Logs</b>     | <code>console.log</code> statements in production code | Remove before deployment                         |
| <b>Error Handling</b> | Inconsistent error display patterns                    | Create shared <code>ErrorBanner</code> component |
| <b>Loading States</b> | Simple text-based loading                              | Implement skeleton loaders consistently          |
| <b>Type Safety</b>    | No TypeScript or PropTypes                             | Add PropTypes or migrate to TypeScript           |

| Area             | Issue                                                  | Recommendation                                                     |
|------------------|--------------------------------------------------------|--------------------------------------------------------------------|
| Testing          | No test files found                                    | Add unit tests for hooks, integration tests for pages              |
| Code Duplication | Avatar color/initials logic repeated in multiple files | Extract to shared utility                                          |
| State Management | Context API only for auth                              | Consider Zustand or Redux for complex state                        |
| Performance      | No memoization on expensive computations               | Use <code>useMemo</code> and <code>useCallback</code> where needed |
| Accessibility    | Limited ARIA attributes                                | Add proper ARIA labels and keyboard navigation                     |
| Documentation    | No JSDoc comments                                      | Add JSDoc to hooks and utility functions                           |

## Scalability Assessment

The architecture supports scaling through:

- Modular page structure (easy to add new modules)
- Custom hooks pattern (easy to add new API integrations)
- Configuration-driven UI (easy to modify dashboards)
- Clear role separation (easy to add new roles)

## 22. README.md

The following is the generated README content:

```
# 🏥 MediFlow – Hospital Management System

! [React] (https://img.shields.io/badge/React-19-blue?logo=react)
! [Vite] (https://img.shields.io/badge/Vite-6-646CFF?logo=vite)
! [Tailwind CSS] (https://img.shields.io/badge/Tailwind-4-06B6D4?logo=tailwindcss)
! [License] (https://img.shields.io/badge/License-MIT-green)
! [Status] (https://img.shields.io/badge/Status-Active-brightgreen)

A comprehensive, role-based Hospital Management System frontend built with React, Vite, and Tailwind CSS. MediFlow streamlines hospital operations for administrators, doctors, and receptionists.

---

## ✨ Features

### 🔒 Authentication
```

- JWT-based authentication with access & refresh tokens
- Role-based route protection (Admin, Doctor, Receptionist)
- Forgot password flow

### ### 👤 Admin Dashboard

- System-wide statistics and activity reports
- Staff account management (create doctors & receptionists)
- Pending account approval workflow
- Doctor schedule management
- User & patient management

### ### 👨 Doctor Dashboard

- Appointment tracking with status updates
- Patient detail management (vitals, medications, notes)
- Health report viewing
- Calendar integration
- Profile settings

### ### 📋 Receptionist Dashboard

- Patient registration with full demographics
- Appointment booking, editing & cancellation
- Real-time queue management with check-in
- Search, filter & pagination across all records
- Quick action shortcuts

---

## ## 🛠️ Tech Stack

| Technology                      | Purpose                   |
|---------------------------------|---------------------------|
| -----                           | -----                     |
| <b>**React 19**</b>             | UI component library      |
| <b>**Vite 6**</b>               | Build tool & dev server   |
| <b>**Tailwind CSS 4**</b>       | Utility-first styling     |
| <b>**React Router v7**</b>      | Client-side routing       |
| <b>**Recharts**</b>             | Dashboard charts & graphs |
| <b>**React Icons (Lucide)**</b> | Icon system               |
| <b>**Context API**</b>          | Global state management   |
| <b>**REST API**</b>             | Backend communication     |

---

## ## 📦 Installation

### ### Prerequisites

- Node.js v18+
- npm v9+
- Backend API server running

### ### Setup

```
```bash
# Clone the repository
git clone https://github.com/Nader-Gamal3011/Nader_Portfolio.git
```

```
# Navigate to project
cd final-project

# Install dependencies
npm install

# Create environment file
cp .env.example .env
```

## Environment Variables

Create a `.env` file in the root directory:

```
VITE_API_BASE_URL=http://localhost:5000/api
VITE_DEFAULT_AVATAR_URL=
VITE_DEFAULT_PATIENT_PHOTO=
VITE_APP_NAME=MediFlow
VITE_APP_VERSION=1.0.0
VITE_APP_DESCRIPTION=Hospital Management System
```

---

## Running the Project

```
# Development server
npm run dev

# Production build
npm run build

# Preview production build
npm run preview

# Lint check
npm run lint
```

---

## API Overview

The frontend communicates with a REST API at the configured `VITE_API_BASE_URL`.

### Key Endpoints

| Module | Endpoint                         | Method | Description          |
|--------|----------------------------------|--------|----------------------|
| Auth   | <code>/auth/login</code>         | POST   | User login           |
| Auth   | <code>/auth/refresh-token</code> | POST   | Refresh access token |

| Module       | Endpoint                                | Method   | Description               |
|--------------|-----------------------------------------|----------|---------------------------|
| Doctor       | <code>/doctors/stats</code>             | GET      | Doctor dashboard stats    |
| Doctor       | <code>/doctors/appointments</code>      | GET      | Doctor appointments       |
| Receptionist | <code>/receptionist/stats</code>        | GET      | Reception dashboard stats |
| Receptionist | <code>/receptionist/patients</code>     | GET/POST | Patient CRUD              |
| Receptionist | <code>/receptionist/appointments</code> | GET/POST | Appointment CRUD          |
| Receptionist | <code>/receptionist/queue</code>        | GET      | Queue management          |
| Admin        | <code>/admin/doctors</code>             | POST     | Create doctor             |
| Admin        | <code>/admin/receptionists</code>       | POST     | Create receptionist       |

## Deployment

### Vercel / Netlify

1. Connect your Git repository
2. Build command: `npm run build`
3. Output directory: `dist`
4. Add environment variables
5. Deploy

### Railway

1. Connect repository
2. Add environment variables
3. Auto-deploys on push

## Project Structure

```
src/
├── components/    # Reusable UI components
├── config/        # Configuration arrays
├── context/       # React Context (Auth)
├── hooks/         # Custom API hooks
├── pages/         # Route-level pages
│   ├── Admin/    # Admin module
│   ├── Doctor/   # Doctor module
│   └── Reception/ # Receptionist module
├── styles/        # Additional CSS
└── utils/         # API client & helpers
```

## License

This project is licensed under the MIT License.

---

## Author

### **Nader Gamal**

- GitHub: [@Nader-Gamal3011](#)

---

\*Documentation generated based on complete codebase analysis of the MediFlow Hospital Management System frontend.\*